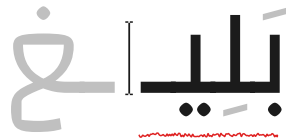


Cairo University
Faculty of Engineering
Department of Computer Engineering



Baligh

Arabic Writing Assistant

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Somia Saad Esmail Elshemy

Supervised by

Dr. Ayman AboElhassan

July, 2026

All rights reserved. This report may not be reproduced in whole or in part, by photocopying or other means, without the permission of the authors/department.

Abstract

Arabic digital writing still suffers from a shortage of integrated tools that can correct Modern Standard Arabic text accurately, respond in real time, and explain their feedback in a way that helps users learn. Existing tools often focus on surface spelling fixes or black-box suggestions without exposing the grammatical reasoning behind the correction. Baligh addresses this problem by providing an Arabic writing assistant that combines grammatical error detection, grammatical error correction, spelling-aware editing, next-word suggestion, and explanatory feedback in a single user-facing system. The project aims to support students, educators, and Arabic content writers by improving writing quality while also making the correction process more understandable and educational.

The implemented system follows a modular pipeline. A preprocessing layer performs normalization, word-boundary handling, segmentation, and morphological analysis and disambiguation. On top of that, Baligh integrates a grammatical error detection engine that fuses rule-based, lexicon-based, and machine-learning detectors, then passes its findings to correction components built around ontology-guided reasoning, dictionary lookup, and text-editing models. In parallel, a next-word suggestion module supports both next-word prediction and word auto-completion. The final system highlights issues, proposes corrections, returns ranked suggestions, and presents grammar-oriented explanations to the user in an interactive workflow.

The project outputs include the preprocessing pipeline, grammatical error detection and correction services, the next-word suggestion subsystem, and a working user application. Testing and validation relied on automated unit and integration tests together with evaluation on established Arabic linguistic resources, including QALB14, QALB15, and ZAEBUC. Overall, Baligh demonstrates that a modular and explainable Arabic writing assistant can combine linguistic processing and machine learning into a practical platform for improving Arabic writing quality.

Contents

Abstract	i
List of Figures	iii
List of Abbreviations	iv
1 Introduction	1
1.1 Project Overview	1
1.2 Individual Role Overview	1
1.3 Document Organization	1
2 Personal Role and Responsibilities	2
2.1 Team Responsibilities and Assigned Tasks	2
2.2 Timeline and Deliverables	2
3 Knowledge Acquisition and Technical Preparation	3
3.1 Investigated Papers	3
3.2 Self-Learning Materials	3
4 Individual Contributions	4
4.1 System Architecture Context	4
4.2 Contribution 1: GEC Edit Tagger (Text Editing Sub-module)	5
4.2.1 Functional Overview	5
4.2.2 Modular Decomposition	7
4.2.3 Design Constraints	10
4.2.4 Results	11
4.3 Contribution 2: Ranker Sub-module	11
4.3.1 Functional Description	12
4.3.2 Modular Decomposition	12
4.3.3 Design Constraints	13
4.4 Contribution 3: Backend Integration (GEC Wrapper & Baligh Wrapper) .	13
4.4.1 The GEC Wrapper	13
4.4.2 The Baligh Wrapper	14

5	Challenges, Lessons Learned, and Future Work	15
5.1	Faced Challenges	15
5.2	Lessons Learned	15
5.3	Future Enhancements	15
A	Edit Tagger Tagset Reference	18
A.1	Core Editing Operations	18
A.2	Operation Compression	18

List of Figures

4.1	High-level modular architecture of the Baligh system.	5
4.2	Overview of the sequence-tagging text editing approach.	6
4.3	Tagger Pre-processing Pipeline	9
4.4	Tagger Model Architecture	9
4.5	Tagger Inference Pipeline	10
4.6	Edit Tagger Correction Example (2)	11
4.7	Edit Tagger Correction Example (1)	11
4.8	Edit Tagger Correction Example (4)	11
4.9	Edit Tagger Correction Example (3)	11

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRF	Conditional Random Field
GEC	Grammatical Error Correction
GED	Grammatical Error Detection
GUI	Graphical User Interface
MSA	Modern Standard Arabic
NLP	Natural Language Processing
NWP	Next-Word Prediction
POS	Part of Speech
QALB	Qatar Arabic Language Bank

Chapter 1: Introduction

1.1. Project Overview

Baligh is an Arabic writing assistant designed to support Modern Standard Arabic writing through explainable correction and real-time assistance. It combines grammatical error detection, correction, automatic vowelization, and next-word suggestion in an interactive workflow.

1.2. Individual Role Overview

My primary role in the Baligh project focused on the Grammatical Error Correction (GEC) components and the overall backend orchestration. Specifically, I was responsible for designing and implementing the GEC Edit Tagger (a sequence-tagging text editing model) and the Ranker Sub-module, which selects the optimal corrections. Furthermore, I architected the GEC Wrapper and the Baligh Wrapper, integrating the independent analysis engines into a cohesive, low-latency pipeline ready for the client application.

1.3. Document Organization

Chapter 2 outlines my specific responsibilities and deliverables. Chapter 3 discusses the technical preparation and knowledge acquisition required for this work. Chapter 4 forms the core of this booklet, detailing my technical contributions including the system architecture context, the Edit Tagger, the Ranker, and the Wrapper integrations. Finally, Chapter 5 reflects on the challenges faced, lessons learned, and potential future improvements.

Chapter 2: Personal Role and Responsibilities

2.1. Team Responsibilities and Assigned Tasks

In a complex NLP pipeline like Baligh, error detection must be paired with robust, context-aware correction. While other team members focused on detecting errors (GED) and providing rule-based fixes, my task was to build a GEC model that could propose edits dynamically, particularly where static rules fall short. Additionally, I was responsible for the crucial task of scoring and selecting the best edits (the Ranker) from multiple sources (Edit Tagger, dictionaries, ontologies), and ensuring these sub-components communicated efficiently via API wrappers (the GEC and Baligh Wrappers).

2.2. Timeline and Deliverables

My work progressed through several phases:

- **Phase 1:** Researching text-editing approaches for GEC and establishing character-to-token edit projection logic.
- **Phase 2:** Developing the sequence-tagging Edit Tagger and its underlying data pipelines.
- **Phase 3:** Designing the Ranker to enforce grammatical accuracy and promote explainable corrections over black-box predictions.
- **Phase 4:** Encapsulating the modules within the GEC Wrapper and the top-level Baligh Wrapper to serve real-time API requests.

Chapter 3: Knowledge Acquisition and Technical Preparation

3.1. Investigated Papers

The literature review focused on text editing grammatical error correction. The work of Alhafni et al. clarified the value of treating Arabic GEC as a sequence-labeling task. [1].

3.2. Self-Learning Materials

Building the GEC components required an extensive review of existing Arabic NLP literature. Traditional Seq2Seq models often suffer from slow inference and lack interpretability, as they generate text autoregressively. This led me to research sequence-tagging (text-editing) architectures, where a model predicts token-level edit operations (KEEP, DELETE, REPLACE, INSERT). I studied alignment algorithms, specifically the Levenshtein edit distance, to map parallel corpora into discrete edit tags. I also familiarized myself with Transformer architectures (e.g., BERT, CAMELBERT) to understand how sub-word tokenization impacts edit projection. For the integration phase, I gained expertise in asynchronous Python programming to ensure the Baligh Wrapper could handle concurrent sub-module execution without bottlenecks. To implement the project effectively, I relied on a mix of coursework, research papers, library documentation, experimentation notebooks, and framework references. The main self-learning inputs were:

- **College Machine Learning course by Dr. Yahia Zakaria:** strengthened the theoretical background behind the statistical modeling side of the project.
- **College Natural Language course by Dr. Sandra Wahid:** reinforced the language-processing concepts most relevant to Arabic NLP work.
- **Arabic NLP pipeline study:** focused on the preprocessing and morphology-analysis workflow used in the project, especially segmentation, disambiguation, and morphological feature access.

Chapter 4: Individual Contributions

4.1. System Architecture Context

To understand where my modules fit, it is important to look at the high-level system architecture of Baligh.

Data enters the system through the **Client Layer**, which consists of the user interface (web text editor) and the REST API backend. The raw text is immediately sent to the **Preprocessing Pipeline**. This pipeline performs foundational text cleaning, normalization, tokenization via Farasa, and contextual morphological analysis and disambiguation using CAMEL tools.

Once the text is preprocessed, it is dispatched into the **Core Features Layer**, which splits into two parallel tracks:

1. **The Slow Path (GEC & GED):** Preprocessed text is fed into the Grammatical Error Detection (GED) Engine. These findings are passed to the Grammatical Error Correction (GEC) Module, which proposes corrections via an ontology-guided grammar lookup, dictionary spelling checks, and a text-editing machine learning model (the Edit Tagger). Each module generates human-readable explanations. A final fusion and ranking layer (the Ranker) consolidates the corrections and explanations.
2. **The Fast Path (NWS):** For predictive typing, the Next-Word Suggestion (NWS) module operates with minimal latency.

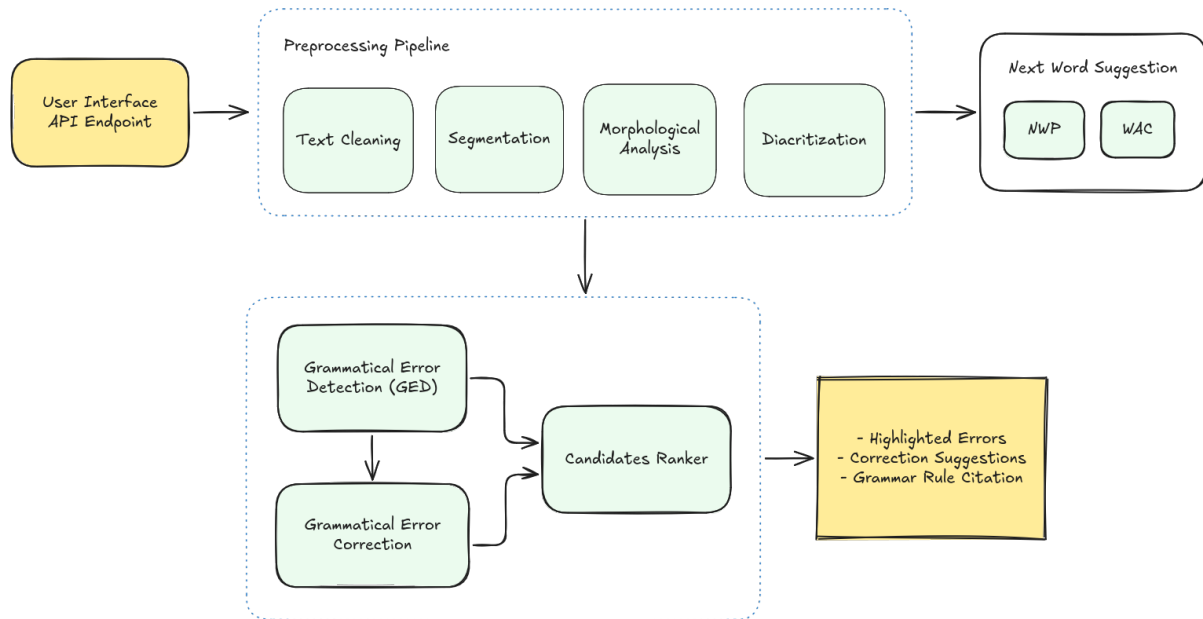


Figure 4.1: High-level modular architecture of the Baligh system.

4.2. Contribution 1: GEC Edit Tagger (Text Editing Sub-module)

4.2.1. Functional Overview

The system implements a Grammatical Error Correction (GEC) model for Arabic using a **text editing (sequence tagging)** approach rather than a traditional sequence-to-sequence (Seq2Seq) generative model. By treating correction as a sequence of edit operations rather than generating text from scratch.

Correction Generation

Instead of generating the corrected sentence autoregressively, the model predicts a sequence of edit operations for each token in the erroneous input. These predicted edits are then applied to the original text to produce the corrected sentence.

Corrected	النفسية . <i>Alnfsyh</i>	الصححة <i>AlSHh</i>		سيما <i>symA</i>	ولا <i>wlA</i>	بالصححة <i>bAlSHh</i>	الاهتمام <i>AlAhtmAm</i>	يجب <i>yjb</i>	(a)				
	7	6	5	4	3	2	1	0					
Erroneous	النفسية <i>Alnfsyh</i>	الصححة <i>AlSHh</i>	في <i>fy</i>	ولاسيما <i>wlAsymA</i>	لصححة <i>lSHh</i>	ب <i>b</i>	الإهتمام <i>AlĀhtmAm</i>	يجب <i>yjb</i>	(b)				
Word Edits	KKKKKKR_[ة]A_[.]	KKKKR_[ة]	DD	KKKI_[]KKKK	MI_[']KKKR_[ة]	K	KKR_[']KKKKK	KKK	(c)				
Word Edits (Compressed)	K*R_[ة]A_[.]	K*R_[ة]	D*	KKKI_[]K*	MI_[']K*R_[ة]	K*	KKR_[']K*	K*	(d)				
	7b	7a	6b	6a	5	4	3b	3a	2	1b	1a	0	
Tokenized Erroneous	ه## ##h	النفسى <i>Alnfsy</i>	ه## ##h	الصح <i>AlSH</i>	في <i>fy</i>	ولاسيما <i>wlAsymA</i>	ه## ##h	لصح <i>lSH</i>	ب <i>b</i>	##htmAm	الإ <i>AlĀ</i>	يجب <i>yjb</i>	(e)
Subword Edits	R_[ة]A_[.]	KKKKKK	R_[ة]	KKKK	DD	KKKI_[]KKKK	R_[ة]	MI_[']KKK	K	KKKKK	KKR_[']	KKK	(f)
Subword Edits (Compressed)	R_[ة]A_[.]	K*	R_[ة]	K*	D*	KKKI_[]K*	R_[ة]	MI_[']K*	K*	K*	K*R_[']	K*	(g)

Figure 4.2: Overview of the sequence-tagging text editing approach.

- **Edit Extraction & Alignment:** The system aligns erroneous and corrected sentence pairs using a weighted Levenshtein edit distance, accompanied by an iterative algorithm to capture multi-word errors. It then applies the character-level alignments.
- **Applying Operations:** It assigns specific character-level edits to the input words to transform them. The primary operations include:
 - Keep (K)
 - Delete (D)
 - Insert (I_[])
 - Replace (R_[])
- **Optimization Strategies:** To keep the vocabulary of possible edits manageable and improve learning, the system uses:
 - **Edit Compression:** Merging consecutive operations (e.g., consecutive deletes DDD into D* or insertions into I_[] I_[*]).
 - **Sub-word Projection:** Edit operations are first extracted at the word level, where grammatical corrections are naturally defined. These edits are then projected onto the corresponding sub-word tokens so that each token receives a label, matching the tokenization scheme used by Transformer models such as BERT.
 - **Edit Segregation:** Splitting the correction into a two-step pipeline—one model for non-punctuation errors, and a subsequent model for punctuation errors.

- **Edit Pruning:** Removing rare edit operations during training to help the model focus on frequent and informative edits.

Explanation Generation (Interpretability)

Unlike autoregressive Seq2Seq models, which generate only the final corrected sentence, edit-based models explicitly predict the edit operation applied to each input token (e.g., **KEEP**, **DELETE**, **REPLACE**, or **INSERT**). These edit labels provide a transparent record of how the input was transformed, making it straightforward to identify the exact grammatical changes performed. This improves the interpretability of the correction process and facilitates error analysis, debugging, and the generation of user-friendly explanations for the applied corrections.

4.2.2. Modular Decomposition

- **Alignment Module**

The aligner is responsible for aligning the erroneous sentence with its corrected one using the Levenshtein edit distance algorithm. The alignment is performed in two stages. First, alignment is carried out at the **word level** to determine the correspondence between words in the two sentences. At this stage, operations such as **insertions**, **deletions**, **merges**, and **splits** are identified to handle cases where a single word in one sentence corresponds to multiple words in the other, or vice versa. Once the word-level alignment has been established, each aligned word pair is further aligned at the **character level** to determine the exact character insertions, deletions, and replacements required to transform the erroneous word into its corrected form. This hierarchical alignment provides the detailed edit information that serves as the basis for extracting edit labels used during model training.

- **Rewrite Module**

After the initial alignment, the rewriter generates an intermediate representation of the aligned sentences. This step resolves ambiguities in the character-level alignment and rewrites complex edits into a consistent form that can be more easily projected onto tokens.

- **Edit Projection Module**

Since grammatical edits are initially extracted at the character level while Transformer-based models operate on tokenized input, the projector maps the character-level edit operations onto the corresponding tokens or sub-word tokens. Each token is assigned an edit label describing the operation that should be applied, making the extracted edits compatible with the model's token-level prediction mechanism.

- **Edit Compression Module**

The projector may produce multiple consecutive edit operations for a single token. The compressor combines these individual operations into a compact edit label whenever possible. This reduces redundancy in the label sequence, simplifies the classification problem, and produces a more manageable set of edit tags for model training and inference.

- **Edit Pruning Module**

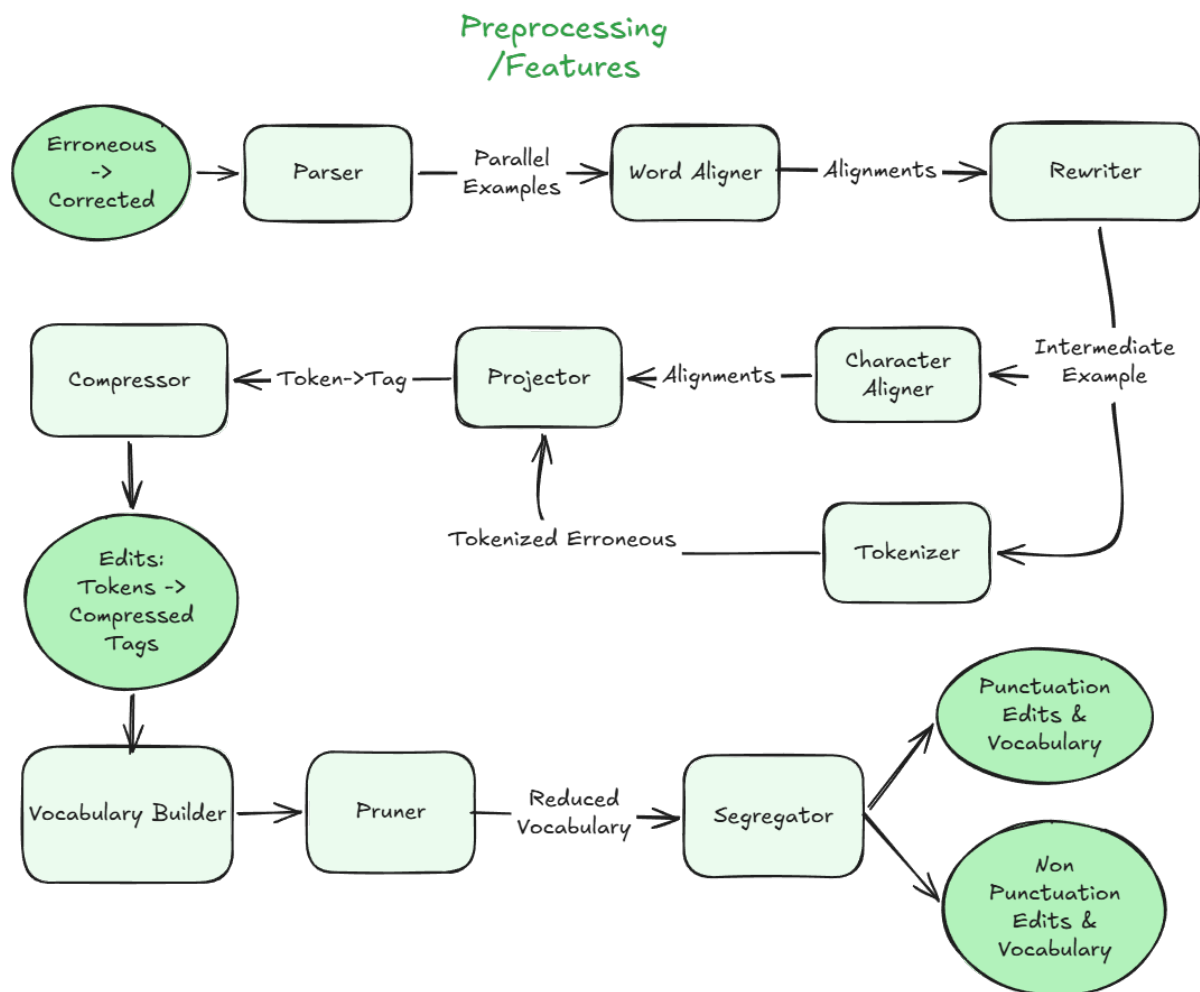
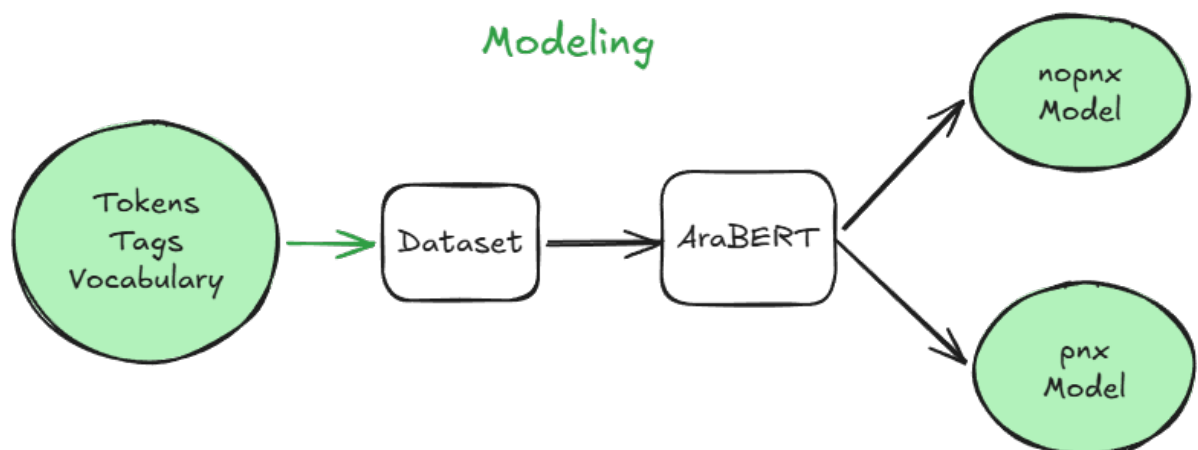
The pruner analyses the frequency of all generated edit labels across the training corpus and removes edit labels that occur below a predefined frequency threshold. Rare edit operations contribute little to the learning process while significantly increasing the number of output classes. Pruning these infrequent labels reduces label sparsity, decreases vocabulary size, and allows the model to focus on learning the most common and informative correction patterns.

- **Edit Segregation Module**

Punctuation corrections often exhibit different characteristics from grammatical and spelling corrections. The segregator separates punctuation-related edit labels from non-punctuation edits, allowing them to be processed independently. This separation improves the handling of punctuation corrections and enables the system to treat punctuation edits differently during training or inference if required.

- **Edit Application Module**

During inference, the model predicts a sequence of edit labels rather than generating corrected text directly. The edits applicator interprets these predicted labels and applies the corresponding operations—such as insertions, deletions, replacements, merges, or splits—to the original input tokens. The resulting sequence of modified tokens is then reconstructed to produce the final corrected sentence.

**Figure 4.3:** Tagger Pre-processing Pipeline**Figure 4.4:** Tagger Model Architecture

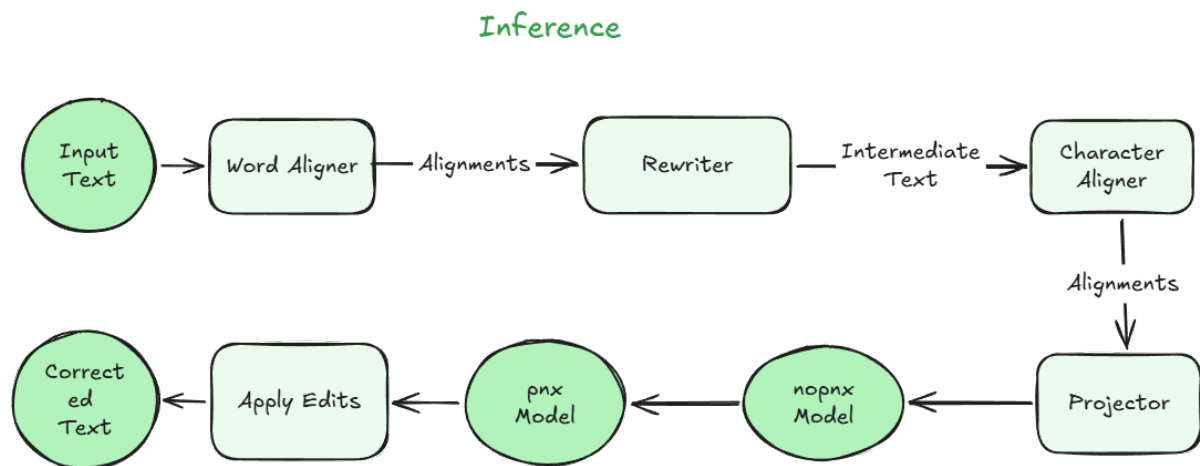


Figure 4.5: Tagger Inference Pipeline

4.2.3. Design Constraints

- The edit tagging task exhibits a significant class imbalance, as the majority of tokens in the training corpus do not require correction and are therefore assigned the KEEP label. This imbalance may bias the model toward predicting KEEP more frequently than edit operations, potentially reducing its ability to detect less common grammatical errors.
- The frequency of grammatical error types in the training corpus is highly uneven. Errors related to Hamza (إ, ء, َ), Taa' Marbuta (ة, ة), and punctuation constitute a large proportion of the annotated corrections, whereas many other grammatical phenomena occur much less frequently. These corpus characteristics directly influence the model's learning behavior. Frequent edit operations are learned more reliably due to their abundance in the training data, while infrequent grammatical phenomena remain more challenging to detect and correct.

4.2.4. Results



Figure 4.6: Edit Tagger Correction Example (2)

المدرسة هي ملقَى الطلاب

Figure 4.7: Edit Tagger Correction Example (1)



Figure 4.8: Edit Tagger Correction Example (4)

يجب ال إهتمام بالصحة

Figure 4.9: Edit Tagger Correction Example (3)

4.3. Contribution 2: Ranker Sub-module

4.3.1. Functional Description

The Ranker module is responsible for producing the final coherent text alongside its supporting grammatical explanations.

- **Correction Generation** evaluates the corrections coming from GEC making use of the GED output. It applies a rigorous filtering and scoring pipeline to candidates from multiple modules. Overlapping or conflicting candidate edits are resolved by selecting the candidate with the highest composite score.
- **Explanation Generation** is facilitated by hardcoded scoring biases that favour explainable, rule-based corrections. Specifically, if a detected error is marked as `explanation_eligible` and the candidate correction comes from the rule-based ONTOLOGY module, a positive weight (`W_EXPLAIN: +0.05`) is applied. This bias guarantees that the final selected corrections can be mapped back to transparent linguistic rules rather than relying solely on black-box predictions.

4.3.2. Modular Decomposition

This module is decomposed into four primary submodules:

1. Configuration Manager

Manages the tunable hyper-parameters used during scoring. Key configurations include base module weights (`W_ONTOLOGY: 0.25`, `W_TAG: 0.15`, `W_DICTIONARY: 0.10`), confidence weightings (`W_EDIT_CONF: 0.30`), and penalty weights (`W_CHAR_DIST: 0.10`, `W_LOW_CONF: 0.15`).

2. Candidate Aggregator & Matcher

Maps candidate edits to specific errors detected by the GED by checking span intersections. Candidates whose correction equals the original text are immediately filtered out.

3. Scoring Engine

Computes a comprehensive `score` for each candidate edit using multiple weighted heuristics:

- **Base Module & Confidence:** Adds base module weight, edit confidence (`W_EDIT_CONF`), and GED confidence (`W_GED_CONF`).
- **Provenance Tiers:** Boosts scores for `tier_1_rule_derived` (+0.15) and `tier_2_rule_supported` (+0.08) error spans.
- **Linguistic Alignment:** Awards +0.10 (`W_SPELL_DICT`) if a Dictionary module corrects an ORTHOGRAPHY error, and +0.10 (`W_GRAM_ONT`) if the Ontol-

ogy module corrects a SYNTAX or MORPHOLOGY errors. Out-of-vocabulary (`is_oov`) tokens receive a +0.05 bonus.

- **Multi-Source Agreement:** Awards +0.20 (`W_AGREEMENT`) for each peer module that suggests the exact same correction string.

4. Final Selection & Metadata Generator

Sorts the scored candidates per error using a strict tuple key: (`score`, `edit_confidence`, `module_priority`, `-length_of_correction`). `module_priority` is strictly ranked: Ontology (3) > Tag (2) > Dictionary (1). It then iterates chronologically through the text, applying the best candidate. Finally, it emits the final correction.

4.3.3. Design Constraints

- **Grammatical Accuracy and Precision:**

The system penalizes edits that diverge too much from the original text. A penalty (`W_CHAR_DIST`: -0.10) is applied proportionately to the normalized Levenshtein distance, and another penalty (`W_LENGTH_RATIO`: -0.05) is applied for deviations in string length. Furthermore, if a candidate's `edit_confidence` falls below `CONF_LOW_THRESHOLD` (0.30), a flat penalty of -0.15 (`W_LOW_CONF`) is applied.

4.4. Contribution 3: Backend Integration (GEC Wrapper & Baligh Wrapper)

Beyond developing individual analytical models, a significant portion of my work involved integrating the dispersed NLP components into a unified, high-performance backend. This was achieved through two critical orchestration layers: the GEC Wrapper and the Baligh Wrapper.

4.4.1. The GEC Wrapper

The GEC Wrapper serves as the unified interface for all correction submodules. Before this wrapper existed, the Ontology, Dictionary, and Edit Tagger modules operated in isolation. The wrapper encapsulates them, managing the asynchronous dispatch of preprocessed text and GED findings to each module concurrently. It collects the varying correction candidates, standardizes their formats into a uniform `CorrectionCandidate` schema, and feeds them directly into the Ranker. This abstraction guarantees that adding or removing future correction models will not break the downstream application logic.

4.4.2. The Baligh Wrapper

The Baligh Wrapper sits at the highest level of the Core Features Layer. It is the master orchestrator that receives requests directly from the REST API. Its responsibilities include:

- **Pipeline Coordination:** Routing text through the Preprocessing Pipeline, then fanning out to the GED and NWS modules, and subsequently passing GED results into the GEC Wrapper.
- **Latency Management:** Utilizing Python's `asyncio` to ensure that heavy machine-learning inferences (like the Edit Tagger) do not block faster rule-based or dictionary lookups.

Chapter 5: Challenges, Lessons Learned, and Future Work

5.1. Faced Challenges

One of the major challenges was dealing with Arabic's morphological richness during edit projection. Aligning character-level edits to sub-word tokens accurately required handling complex affixes and clitics seamlessly. Another significant challenge was latency tuning in the Baligh Wrapper. Balancing the heavy computational requirements of sequence-tagging models against the strict real-time constraints of a web editor required careful asynchronous design and payload minimization.

5.2. Lessons Learned

- **Interpretability Matters:** Moving from generative Seq2Seq to an edit-tagging approach proved that making AI explainable is just as important as making it accurate. Users trust corrections more when they can see exactly what changed.
- **The Benefit of Good Communication and Documentation:** Clear interfaces between modules (e.g., the standardized `CorrectionCandidate` schema) prevented integration breakdowns across the team.

5.3. Future Enhancements

Future iterations could explore replacing the hard-coded heuristic Ranker with a lightweight machine-learning ranker trained specifically on user acceptance rates. Additionally, the GEC Wrapper could be optimized further with model quantization or ONNX runtime integration to reduce the memory footprint of the Edit Tagger, improving the overall throughput of the Baligh Wrapper. The edit vocabulary could also benefit from more aggressive compression strategies and better handling of underrepresented grammatical phenomena in the training corpus. Training on More Diverse Corpora: The current model was trained on corpora in which certain error categories, such as Hamza, Taa' Marbuta, and punctuation mistakes, are significantly more frequent than others. As a result, the model is naturally more proficient at correcting these common errors. Future

work should focus on training the model using larger and more diverse corpora containing richer grammatical phenomena, including complex syntactic, semantic, and stylistic errors. Expanding the Edit Vocabulary: Some rare edit operations were removed during edit pruning to control the size of the output label space. Future work may explore adaptive or hierarchical edit representations that preserve rare corrections without substantially increasing model complexity.

References

- [1] B. Alhafni and N. Habash, "Enhancing Text Editing for Grammatical Error Correction: Arabic as a Case Study," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

Chapter A: Edit Tagger Tagset Reference

This appendix summarizes the core edit operations predicted by the Grammatical Error Correction Edit Tagger. While the full vocabulary of edits is dynamically extracted from the training corpora and can include thousands of specific REPLACE and INSERT strings, the underlying mechanisms rely on these fundamental operations.

A.1. Core Editing Operations

- **KEEP (K)**: Indicates that the token is grammatically and orthographically correct and should remain unchanged in the final output. This is the most frequently predicted class due to class imbalance.
- **DELETE (D)**: Indicates that the token is extraneous or incorrect and should be completely removed from the sentence.
- **REPLACE (R_[text])**: Instructs the applicator to substitute the current token with the provided string. This is commonly used for spelling corrections, l'rab fixes, and morphological adjustments. Example: R_[] or R_[].
- **INSERT (I_[text])**: Instructs the applicator to insert a missing word or character adjacent to the current token. Example: I_[].

A.2. Operation Compression

To keep the vocabulary size manageable, consecutive identical operations are compressed during the extraction phase and decompressed during application.

- **D***: Represents multiple consecutive deletions.
- **I_[text*]**: Represents complex consecutive insertions that are merged into a single tag.